

DEGREE PROJECT FOR MASTER OF COMPUTER SCIENCE WITH
SPECIALIZATION IN CYBERSECURITY
DEPARTMENT OF ENGINEERING SCIENCE
UNIVERSITY WEST

Automated Compliance Mapping for IoT and AI Security Standards: An NLP-Based Approach for ETSI EN 303 645 and EN 304 223

A case study

Ashish Gurung
Bivishan Dhungana



A THESIS SUBMITTED TO THE DEPARTMENT OF ENGINEERING SCIENCE
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF COMPUTER SCIENCE WITH SPECIALIZATION IN CYBERSECURITY
AT UNIVERSITY WEST
AUGUST 2026

Author(s):	Ashish Gurung Bivishan Dhungana
Supervisor:	Fatiha Djebbar
Examiner:	Eleftherios Vlahakis
Program:	Master Programme in Cybersecurity
Main field of study:	Computer Science
Credits:	15 Higher Education credits
Keywords:	compliance mapping, semantic similarity, large language models, IoT security, AI security
Template:	University West, IV-Master 2.10
Publisher:	University West, Department of Engineering Science S-461 86 Trollhättan, SWEDEN Phone: +46 520 22 30 00 Web: www.hv.se

Summary

Organizations building AI-enabled IoT products must show compliance with several security standards at once, and aligning their overlapping requirements by hand is slow and demands expertise in both domains. This thesis examines how far that alignment can be automated for two ETSI standards: EN 303 645 for consumer IoT security and EN 304 223 for AI system security. A pipeline was built that extracts 141 normative provisions from the two standards (69 and 72 respectively) and maps them to each other with seven methods: TF-IDF similarity, security-keyword matching, three transformer embedding models (Sentence-BERT, BERT, SecureBERT), Gemini text embeddings, and a Gemini LLM prompted to classify each of the 4,968 possible pairs into one of six relationship types. Predictions were evaluated against two reference sets, both built by LLM-assisted annotation: a purposive pilot of 107 pairs reviewed by the authors, and a stratified probability sample of 650 pairs drawn from the full candidate space with known inclusion probabilities.

Results depend sharply on how the reference set is built. Measured on the purposively assembled pilot set, where 79% of pairs are related, BERT reaches a pair-detection F1 of 0.89 — which is also what labelling every pair as related achieves, because general-purpose embedding spaces score nearly all provisions as highly similar. The probability sample puts the true rate of related pairs at 12.4%, and at that rate four of the seven methods score exactly the all-positive baseline, rejecting almost no unrelated pair. Most of that collapse is calibration rather than capability: re-selecting each threshold by cross-validation on the sample lifts Gemini embeddings to F1 0.50 and Sentence-BERT to 0.42 against a baseline of 0.22, differences a paired bootstrap separates from zero, while SecureBERT clears the baseline at no threshold at all. Assigning the correct relationship type is harder still: the best method reaches 0.48 accuracy, matching but not exceeding the majority-class baseline. A separate finding concerns the LLM classifier, whose labels agree with themselves at $\kappa = 0.02$ across two runs on identical input, so its reported scores are not stable properties of the method. The results support automated mapping as a candidate-ranking aid for human reviewers, not as a replacement for expert judgement.

Affirmation

This master degree report, *Automated Compliance Mapping for IoT and AI Security Standards: An NLP-Based Approach for ETSI EN 303 645 and EN 304 223*, was written as part of the master degree work needed to obtain a Master of Computer Science with specialization in Cybersecurity degree at University West. All material in this report that is not our own is clearly identified and used in an appropriate and correct way. The main part of the work included in this degree project has not previously been published or used for obtaining another degree.

Ashish Gurung

Date

Bivishan Dhungana

Date

Contents

Summary	iii
Affirmation	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Motivation	1
1.2 Aim	2
1.3 Research questions	2
1.4 Contributions	2
1.5 Outline	3
2 Background and Related Work	4
2.1 Background	4
2.1.1 IoT security and ETSI EN 303 645	4
2.1.2 AI system security and ETSI EN 304 223	4
2.1.3 Compliance mapping	5
2.1.4 NLP and LLM techniques for requirement analysis	5
2.2 Related work	6
2.2.1 Automated compliance mapping in cybersecurity	6
2.2.2 NLP and semantic techniques for requirement mapping	6
2.2.3 LLM-based compliance analysis	6
2.2.4 Positioning of this thesis	6
3 Research Methodology	8
3.1 Research design	8
3.2 Data: standards and provision extraction	8
3.2.1 Source documents	8
3.2.2 Extraction of normative provisions	8
3.3 Relationship taxonomy	9
3.4 Ground-truth construction and validation	9
3.4.1 LLM-assisted annotation	9
3.4.2 A probability sample of the candidate space	10
3.4.3 Reliability of the reference set	11
3.5 Automated mapping methods	12
3.5.1 Rule-based baselines	12
3.5.2 Transformer embedding models	12
3.5.3 LLM-based methods	12
3.5.4 From similarity scores to labels	13

3.6	Evaluation framework	13
3.6.1	Pair detection	13
3.6.2	Relationship classification	13
3.6.3	Uncertainty and sensitivity	13
3.6.4	Efficiency and interpretability	14
4	The Mapping Pipeline	15
4.1	Architecture	15
4.2	Stage descriptions	15
4.3	Built versus reused	16
4.4	Reproducibility	16
5	Results and Discussion	17
5.1	Experimental setup	17
5.2	Pair detection	17
5.3	Similarity structure	19
5.4	Threshold sensitivity	19
5.5	Corpus-level performance	20
5.6	Relationship classification	23
5.7	Ground-truth validation	25
5.8	Efficiency and interpretability	26
5.9	Discussion	27
5.10	Threats to validity	27
6	Conclusion	29
	Bibliography	33
	Glossary and Symbols	34
A	Relationship Label Definitions	35
B	Expert Validation Survey Instrument	37
B.1	Purpose	37
B.2	Participants and recruitment	37
B.3	Ethics and data handling	37
B.4	Screening questions	37
B.5	Main task	38
B.6	Post-task questions	38
B.7	Analysis plan	38
B.8	Expected outcomes	38
C	Reproduction Guide	39

List of Figures

3.1	Sampling strata and observed yield	11
4.1	Pipeline stages and the artefact each stage produces. Every stage is a separate command, so intermediate outputs can be inspected before the next stage runs.	15
5.1	Distribution of relationship labels in the 107-pair ground truth.	17
5.2	Pair-detection precision, recall, and F1 per method.	18
5.3	Bootstrap 95% confidence intervals (2,000 resamples) for pair-detection F1 and classification macro-F1.	18
5.4	Distributions of cosine similarity for annotated related pairs (blue) and annotated negatives (orange), per embedding model.	19
5.5	Pair-detection F1 as a function of the similarity threshold, computed from the stored similarity matrices on the annotated scope.	19
5.6	Cross-validated threshold calibration	20
5.7	Precision as a function of prevalence	22
5.8	Effect of recalibrating the operating point	23
5.9	Row-normalised confusion matrices (counts printed) for the three most instructive methods. EQV = equivalence, OVL = overlap, SUB = subsumption, CMP = complementarity, NONE = no relation.	24

List of Tables

3.1	Stratified sampling design over the 4,968 candidate pairs. Unequal allocation concentrates annotation where relations are common while keeping every inclusion probability strictly positive, which is what keeps the estimator unbiased.	10
5.1	Pair detection on the annotated scope. P^+ is the number of pairs the method marks as related among the 107 annotated pairs. Precision counts false positives only on the 22 annotated negatives, which is why sparse methods show precision 1.00 despite very low recall.	18
5.2	Corpus-level detection performance	21
5.3	Corpus performance at a corpus-calibrated operating point	22
5.4	Relationship classification over all 107 annotated pairs, five-label scheme (subsumption directions merged). CIs are bootstrap 95% intervals.	24
5.5	Reference-set agreement against the pilot and the LLM contrast	25
5.6	Wall-clock runtime per stage on a CPU-only laptop (Apple silicon), including model loading. Gemini stages run through the batch API; their end-to-end time is dominated by queueing (minutes to hours) rather than computation.	26

1 | Introduction

Industrial automation is shifting from traditional rule-based control towards systems that incorporate artificial intelligence and, increasingly, agentic behaviour [1]. At the same time, the number of connected IoT devices keeps growing, and each device widens the attack surface: weak authentication, unprotected update channels, and exposure of personal data are recurring problems [2, 3]. Standardization bodies have responded with security baselines for these technologies. Two of them are central to this thesis. ETSI EN 303 645 defines baseline cybersecurity provisions for consumer IoT devices, covering areas such as password policy, secure communication, software updates, and protection of personal data [4]. ETSI EN 304 223, published by the ETSI committee on Securing Artificial Intelligence, defines baseline security requirements for AI models and systems across their life cycle, from secure design to secure end of life [5].

A company that ships an AI-enabled IoT product falls under both documents. Demonstrating compliance then means working out how the two sets of requirements relate: which provisions say essentially the same thing, which overlap partially, which cover different aspects of a shared objective, and which have no counterpart at all. Both standards express their requirements in natural language, at different levels of abstraction and with domain-specific vocabulary, so this mapping work is done manually today. It is slow, error-prone, and requires expertise in both IoT and AI security [6, 7]. The burden grows with every additional regulation — the EU Cyber Resilience Act and the AI Act add their own requirement catalogues on top of the sector standards [8].

Recent progress in natural language processing offers a way to reduce this effort. Embedding models built on transformers can measure semantic similarity between sentences even when the wording differs [9, 10], and large language models can be prompted to judge the relationship between two requirements directly [11]. Whether these techniques are accurate enough for security standards — where precision and traceability matter — is still an open question, and that question is what this thesis investigates.

1.1 Motivation

Compliance mapping sits in an awkward middle ground. It is too language-heavy for classical rule-based tooling, yet too consequential to hand over to an opaque model without evidence of its reliability. Published work on automated compliance analysis reports promising results in neighbouring domains — privacy policies, GDPR data processing agreements, cloud security posture [11, 12, 13] — but the specific task of mapping one security standard onto another has received little systematic evaluation. By implementing a spectrum of methods, from keyword matching to LLM classification, and scoring them against a common expert reference on a real standard pair, this project produces evidence about where automation helps, where it fails, and where human expertise remains indispensable. The results are relevant both to researchers working on NLP for requirements engineering and to practitioners who need to decide whether AI-assisted compliance tools can be trusted in their processes.

1.2 Aim

The project builds and evaluates an NLP-based pipeline that identifies semantic relationships between the provisions of ETSI EN 303 645 and ETSI EN 304 223. The aim is not to develop new model architectures but to establish, empirically, how well existing techniques perform on this task. Concretely, the project (i) extracts the normative provisions of both standards into a structured dataset, (ii) applies seven automated mapping methods spanning rule-based, embedding-based, and LLM-based approaches, (iii) evaluates them against an author-reviewed, LLM-assisted ground truth of annotated provision pairs, and (iv) analyses the errors to characterise the limitations of each method family. The outcome is a grounded assessment of the feasibility of automated compliance mapping between IoT and AI security standards, together with a reusable dataset and evaluation framework.

1.3 Research questions

- RQ1.** To what extent can NLP and AI techniques automatically identify semantic relationships (equivalence, overlap, subsumption, complementarity) between security requirements in ETSI EN 303 645 and ETSI EN 304 223, and what are the fundamental challenges and limitations?
- RQ2.** How do different automated approaches — rule-based methods, transformer embedding models (Sentence-BERT, BERT, SecureBERT), and LLM-based methods (Gemini embeddings and Gemini prompt classification) — compare on the compliance mapping task?
- RQ3.** How does automated mapping compare to expert judgement in terms of accuracy, coverage, efficiency, and interpretability?

1.4 Contributions

The main contributions of this degree project are:

- A structured dataset of 141 normative provisions extracted from ETSI EN 303 645 and ETSI EN 304 223, and two reference mappings annotated with six relationship types, justifications, and confidence scores: a purposive pilot of 107 pairs, and a stratified probability sample of 650 pairs with known inclusion probabilities, which permits estimates for the full 4,968-pair candidate space rather than for a hand-picked subset of it.
- An open, reproducible pipeline implementing seven mapping methods — TF-IDF, security-keyword Jaccard matching, Sentence-BERT, BERT, SecureBERT, Gemini embeddings, and Gemini LLM classification — with a common evaluation harness.
- An empirical comparison of these methods on pair detection and relationship classification, including confidence intervals, threshold sensitivity analysis, and qualitative error analysis.
- Evidence-based observations on why general-purpose embedding models overestimate similarity between security requirements, and on the practical role automated mapping can play in compliance work today.

1.5 Outline

Chapter 2 introduces the two standards and reviews related work on automated compliance analysis. Chapter 3 describes the research design: data extraction, the relationship taxonomy, ground-truth construction and validation, the mapping methods, and the evaluation metrics. Chapter 4 presents the implemented pipeline. Chapter 5 reports and discusses the experimental results, and Chapter 6 concludes and outlines future work.

2 | Background and Related Work

This chapter gives the technical and regulatory context for the project. It first introduces the two standards and the security challenges they respond to, then describes compliance mapping as an activity, summarises the NLP and LLM techniques the project builds on, and finally reviews prior studies on automated compliance analysis.

2.1 Background

2.1.1 IoT security and ETSI EN 303 645

Consumer IoT devices are network-connected by design, often built on constrained hardware, and frequently shipped with weak defaults. Recurring weaknesses include poor authentication mechanisms, unprotected software updates, unencrypted communication, unnecessarily exposed interfaces, and insufficient protection of personal data [14, 15]. Because such devices sit in homes and workplaces, even small security failures translate into concrete privacy and safety risks [16].

ETSI EN 303 645 (*Cyber Security for Consumer Internet of Things: Baseline Requirements*) addresses these weaknesses with thirteen groups of provisions, including no universal default passwords, vulnerability disclosure, keeping software updated, secure communication, minimising attack surfaces, and ensuring the protection of personal data [4]. The standard is outcome-focused: it states what a device must achieve rather than prescribing implementations, and it distinguishes mandatory provisions (“shall”) from recommendations (“should”). Its provisions are organised in numbered clauses (e.g., provision 5.3-3 on update mechanisms), which makes it a practical anchor for systematic mapping.

2.1.2 AI system security and ETSI EN 304 223

AI systems face threats that classical software security does not fully cover: training data poisoning, model manipulation and theft, adversarial inputs, and prompt injection in systems built on large models [17, 18]. Security work for AI must therefore address the model life cycle — data collection, training, deployment, maintenance, and disposal — rather than only the running system.

ETSI EN 304 223 (*Securing Artificial Intelligence: Baseline Cyber Security Requirements*) structures its provisions along exactly that life cycle: secure design (clause 5.1), secure development (5.2), secure deployment (5.3), secure maintenance (5.4), and secure end of life (5.5) [5]. Within these phases it covers AI-specific control areas such as threat and risk management, documentation and auditability of models, protection of datasets and prompts, supply-chain security, security testing, behavioural monitoring, and secure disposal of data and models. The provision-based, life-cycle structure parallels the clause organisation of ETSI EN 303 645, which is what makes a pairwise mapping between the two documents feasible in the first place.

2.1.3 Compliance mapping

Compliance mapping is the activity of relating the requirements of one framework to those of another, so that evidence collected for one can be reused for the other and gaps become visible. Studies of security standards report that overlapping controls, inconsistent terminology, and fragmented requirements are common across frameworks, and that mapping them requires interpretation of purpose and security objective, not just wording [7, 19]. Similar observations come from the GDPR domain, where manual compliance checking is described as time-consuming and dependent on scarce legal-technical expertise [12, 20]. Castellanos Ardila et al. [21] likewise find that manual compliance checking of software processes is labour-intensive and hard to scale.

Automated approaches try to reduce this effort by extracting requirements from the documents and comparing them algorithmically, classifying pairs as identical, related, partially overlapping, or unrelated [22]. Beyond saving effort, automation promises consistency: the same criteria are applied to every pair, which manual mapping cannot guarantee. The open question is accuracy, and that is the subject of this thesis.

2.1.4 NLP and LLM techniques for requirement analysis

Semantic similarity. Standards frequently express similar objectives with different terminology — device-focused language in ETSI EN 303 645, life-cycle and governance language in ETSI EN 304 223. Keyword matching fails on such vocabulary mismatch, a problem long recognised in requirements traceability research [23, 24]. Semantic approaches map text into vector spaces where related meanings land close together, and have been shown to outperform lexical matching for requirement tracing [23].

NLP pipelines. NLP has been applied across requirements engineering tasks: extracting domain concepts, detecting language defects, and establishing traceability links [25, 26]. A typical pipeline extracts normative statements, normalises the text, and converts each requirement into a numerical representation suitable for comparison. Legal and regulatory text adds difficulties of its own — context-dependent terminology, implicit meaning, and complex cross-references [27].

Transformer models. Transformer architectures learn contextual representations of language through self-attention. BERT [9] provides bidirectional contextual embeddings that transfer well to many NLP tasks. For sentence-level comparison, Sentence-BERT [10] fine-tunes a Siamese architecture so that cosine similarity between sentence embeddings reflects semantic relatedness, making large-scale pairwise comparison cheap. SecureBERT [28] adapts a RoBERTa-based model to cybersecurity text, motivating its inclusion here as a domain-tuned alternative. One caveat known from the literature is that the embedding spaces of contextual models are anisotropic — vectors concentrate in a narrow cone, so unrelated sentences can still receive high cosine similarity [29]. This property turns out to matter for the results in Chapter 5.

Large language models. LLMs can be prompted to compare two requirements directly and output a relationship judgement, capturing nuances that similarity scores flatten. Reported drawbacks include computational cost, limited reproducibility, and sensitivity to prompt design [30, 11].

Gap analysis. Mapping two standards also serves gap analysis: identifying objectives covered by one document but not the other. ETSI EN 303 645 concentrates on device-level baseline security while ETSI EN 304 223 concentrates on the AI life cycle, so their relationship is expected to consist mostly of partial overlaps and complementary provisions rather than one-to-one equivalences [4, 5]. The task definition therefore covers partial and absent alignment as well as strong matches.

2.2 Related work

2.2.1 Automated compliance mapping in cybersecurity

Bar-Haim et al. [13] automate the assessment of organizational cybersecurity posture in cloud environments by mapping free-text security requirements to control frameworks — a NIST-based framework of 280 controls in 22 families and a finer-grained three-level risk framework of 169 categories. They formulate mapping as supervised text-pair classification with RoBERTa and SBERT models and use hard negative sampling to force the models to separate semantically close but non-equivalent controls. Their best combined model reaches about 72% precision@1 for control-family mapping and 55% for individual controls, degrading as granularity increases. The authors conclude the approach suits a human-in-the-loop workflow rather than autonomous operation — a conclusion this thesis revisits for the standard-to-standard setting.

2.2.2 NLP and semantic techniques for requirement mapping

Sapkota et al. [31] present RegCMantic, a framework that maps regulatory guidelines to organizational processes. Regulatory documents are parsed into a uniform XML structure, requirement entities (subject, obligation, action) are extracted into an ontology, and mapping uses three similarity measures over topics, core entities, and auxiliary entities. In a pharmaceutical case study the extended framework improves extraction F-measure from 0.83 to 0.91 and performs mapping evaluated against expert-created references at a 0.85 similarity threshold. The study demonstrates that ontology-supported NLP can automate requirement mapping, while noting that ambiguity and implicit, domain-specific meaning remain the hard part — the same difficulties observed with the standards studied here.

2.2.3 LLM-based compliance analysis

Rodríguez et al. [11] investigate LLMs for large-scale privacy policy analysis, comparing ChatGPT and Llama 2 against earlier symbolic and machine-learning baselines on the MAPP and OPP-115 datasets. With a two-shot prompt design their best configuration exceeds 93% F1, outperforming prior techniques while requiring less annotated data and engineering effort. The study is limited to privacy policies, but it provides strong evidence that prompted LLMs can support compliance-oriented text analysis, motivating the inclusion of an LLM classification method in this project.

2.2.4 Positioning of this thesis

Prior work automates mapping between requirements and control frameworks [13], between regulations and business processes [31], and classification within privacy policies [11]. Mapping two security standards to each other — with a typed relationship taxonomy rather than a binary match — has not, to our knowledge, been evaluated systematically

for the IoT/AI standard pair. This project fills that gap with a controlled comparison of seven methods against a common reference.

3 | Research Methodology

3.1 Research design

The project follows an experimental, comparative design. Rather than proposing a new algorithm, it measures how well existing NLP and LLM techniques perform on one concrete task: identifying and typing semantic relationships between the provisions of ETSI EN 303 645 and ETSI EN 304 223. All methods are run on the same extracted dataset and scored against the same ground truth, so differences in the results can be attributed to the methods themselves.

The design addresses the research questions directly. RQ1 (feasibility and limits of automated relationship identification) is answered by the absolute performance levels and the error analysis. RQ2 (comparison of method families) is answered by evaluating all seven methods under identical conditions. RQ3 (automated vs. expert mapping) is answered by treating the reference annotation as the standard of comparison and additionally measuring how consistently the task is performed on repetition. The expert role in that comparison is filled by LLM-assisted annotation reviewed by the authors against a written codebook, not by external domain experts; a protocol for the external study is given in Appendix B but was not carried out, and Section 5.10 states what this costs the RQ3 answer.

The workflow has five stages — extraction, ground-truth construction, automated mapping, evaluation, and reporting — described in the remainder of this chapter and implemented in the pipeline of Chapter 4.

3.2 Data: standards and provision extraction

3.2.1 Source documents

Both standards are publicly available from ETSI. ETSI EN 303 645 v3.1.3 defines baseline security provisions for consumer IoT devices; ETSI EN 304 223 v2.1.1 defines baseline security requirements for AI models and systems. The two documents differ in structure and terminology but share a numbered-provision format, which the extraction step exploits.

3.2.2 Extraction of normative provisions

Security standards mix normative requirements with explanatory prose, examples, and notes. Only the normative statements are relevant for mapping, so the first step isolates them. The PDF documents are converted to plain text with `pdftotext`, recurring page headers and footers are stripped, and provisions are located by their clause identifiers (e.g., “Provision 5.3-3” in ETSI EN 303 645, numbered requirement clauses in ETSI EN 304 223). For each provision the pipeline records: the provision identifier, source standard, section number and title, the full provision text, and its modality — whether the provision uses *shall* (mandatory), *should* (recommended), or *may* (permitted).

The extraction yields 69 provisions from ETSI EN 303 645 and 72 from ETSI EN 304 223, 141 in total. Since any provision of one standard could in principle relate to

any provision of the other, the mapping space contains $69 \times 72 = 4,968$ candidate pairs. Extraction correctness was checked by sampling provisions and comparing them against the original PDFs; counts per section were also compared with the tables of contents of both standards.

Text preprocessing is deliberately minimal. The transformer models used in this project expect natural sentences and apply their own tokenisation, so provisions are passed to them essentially as extracted. Only the TF-IDF baseline applies lowercasing and English stop-word removal as part of its vectoriser. Domain terms (authentication, encryption, integrity, vulnerability, and so on) are never stripped, since they carry the meaning the mapping depends on.

3.3 Relationship taxonomy

Every candidate pair (a, b) , with a from ETSI EN 303 645 and b from ETSI EN 304 223, is assigned exactly one of six relationship types:

EQUIVALENCE

a and b express the same security objective with comparable scope.

SUBSUMPTION (A BROADER)

a covers the objective of b and more; b is a special case of a .

SUBSUMPTION (B BROADER)

the converse: b covers a .

OVERLAP

the provisions share part of their objective, but each also covers ground the other does not.

COMPLEMENTARITY

the provisions address different aspects that jointly support a common security goal, without overlapping scope.

NO RELATION

no meaningful connection.

Full definitions with decision guidance appear in Appendix A. For evaluation, the two directional subsumption labels are merged into a single SUBSUMPTION class, since direction confusions are less interesting than detecting subsumption at all and the per -direction supports are small.

3.4 Ground-truth construction and validation

3.4.1 LLM-assisted annotation

Evaluating the automated methods requires a reference mapping. Annotating all 4,968 pairs manually was outside the scope of a 15-credit project, so the ground truth was produced with LLM assistance and human review, a set-up increasingly common in annotation work and disclosed here for transparency. Candidate pairs and draft relationship labels were generated with Claude Opus 5 (Anthropic), prompted with the full provision texts and the taxonomy definitions of Section 3.3. The authors then reviewed the drafts, corrected labels and justifications, and removed pairs whose connection did not hold up

against the original standard texts. Each retained pair carries a short written justification and a reviewer confidence score from 1 (uncertain) to 3 (certain).

The resulting ground truth contains 107 pairs: 85 positive pairs (3 equivalence, 51 overlap, 9 subsumption across both directions, 22 complementarity) and 22 explicit NO RELATION pairs. The negative pairs were chosen to be non-trivial — provisions that share surface vocabulary but differ in objective — so that methods relying on word overlap are actually tested. The class distribution is skewed towards OVERLAP, which reflects the real relationship between a device-level and a life-cycle-level standard: near-identical requirements are rare.

Using an LLM in the annotation loop creates a risk of circularity, since one of the evaluated methods is itself an LLM. Two design choices reduce this risk: the annotation model (Claude) is from a different family than the evaluated model (Gemini), and the labels follow a codebook written before any were drafted. Human review, however, covers only the 107 pilot pairs. The 650-pair probability sample of Section 3.4.2 was not reviewed pair by pair — at that size it would have consumed the project’s remaining budget — so its reliability rests on the blind comparison against the reviewed pilot reported in Section 3.4.3. The residual risk is examined there and discussed in Section 5.10.

3.4.2 A probability sample of the candidate space

The pilot set of Section 3.4 was assembled purposively, by examining pairs that looked promising. That makes it useful for developing the method and unusable for estimating anything about the corpus: its pairs have no inclusion probabilities, so its 79.4% positive rate cannot be corrected toward the rate a deployed system would meet. A second reference set was therefore drawn as a probability sample.

Pairs were first ordered by a screening score: the mean percentile rank of each pair across all five similarity matrices. Averaging over the methods keeps the strata neutral between them. Stratifying on one model’s scores would instead sample densely wherever that model is confident, and quietly flatter it at evaluation. The ordered space was then cut into three strata and sampled at deliberately unequal rates.

Table 3.1: Stratified sampling design over the 4,968 candidate pairs. Unequal allocation concentrates annotation where relations are common while keeping every inclusion probability strictly positive, which is what keeps the estimator unbiased.

Stratum	Screening rank	N	n	Weight N/n
H	top 10%	497	250	1.99
M	next 30%	1 490	200	7.45
L	bottom 60%	2 981	200	14.91
		4 968	650	

Each pair stores its stratum and weight, so corpus-level precision, recall, specificity and prevalence follow by Horvitz–Thompson estimation, with confidence intervals from a bootstrap that resamples within stratum. Batches are shuffled under a fixed seed before annotation, so a partially completed pass is still a random subsample rather than a biased prefix. All 650 designed pairs were annotated, in 19 batches of 40, together with the 107 pilot pairs mixed into the same stream.

Disproportionate allocation buys efficiency at a price in variance. The weights span 1.99 to 14.91, giving a Kish design effect of 1.49: the 650 pairs carry the information of about 437 drawn with equal probability. This is reported alongside the estimates in Section 5.5

rather than left implicit, since a weighted sample whose nominal size is quoted without its design effect overstates its own precision.

Annotation followed a written codebook fixed before labelling began, reproduced in Appendix A. It states the decision procedure, the overlap-versus-complementarity test that carries most of the difficulty, and one amendment made partway through and applied from that point onward. The 107 pilot pairs were mixed into the stream unmarked, so re-labelling them was blind and their agreement with the earlier author-reviewed labels can be read as a check on the new annotation (Section 5.7).

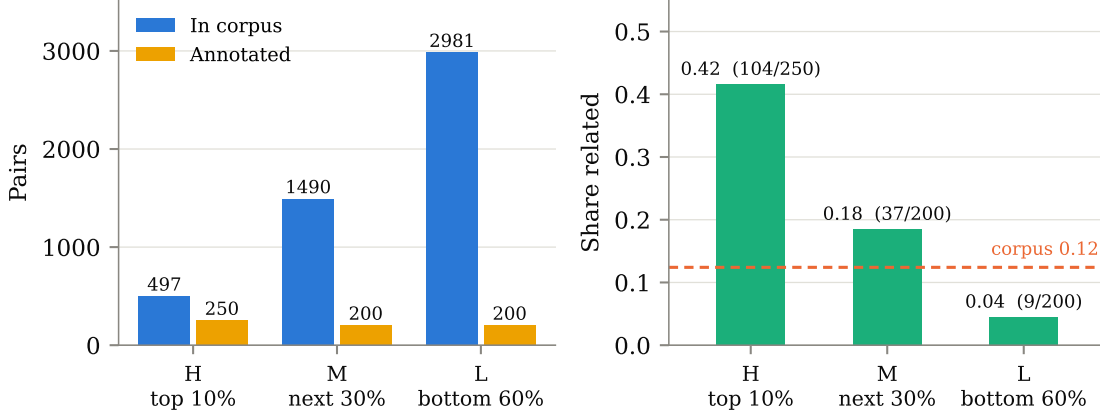


Figure 3.1: Left: size of each stratum in the candidate space against the number of pairs annotated from it. Right: the share of annotated pairs in each stratum carrying a relationship, with the weighted corpus estimate for reference. The monotone gradient across strata is evidence that the screening score orders the space as intended; the corpus estimate sits below all but the lowest stratum because that stratum holds three pairs in five.

Figure 3.1 shows the realised sample. The stratum yields (0.42, 0.18, 0.04) confirm the screening score separates the space, and they are also why the design is efficient: an unstratified sample of the same size would spend three annotations in five on a region where one pair in twenty-five is related.

One class stayed out of reach. The sample contains no EQUIVALENCE pair at all, despite covering half of the top-decile stratum, so no weighted per-class estimate can be given for it; the three instances in the corpus known to the project all come from the purposive pilot. That is itself a measurement — two standards written for different layers of a system produce overlapping and complementary obligations, but almost never identical ones — and it is the reason per-class results in Section 5.6 are reported on the pilot set only.

3.4.3 Reliability of the reference set

Three statistics are reported, all computable from what was actually done. The first is agreement between the sample and the author-reviewed pilot labels on the pairs they share, using raw agreement and Cohen’s kappa [32] with the bands of Landis and Koch [33]. The second is calibration: whether recorded annotation confidence predicts where the two label sets diverge. The third is a contrast rather than a validation — the run-to-run self-agreement of the evaluated LLM on identical input, which establishes what an unreliable annotation source looks like on the same scale.

Cross-model agreement using Gemini is deliberately excluded. Gemini is one of the evaluated methods, so its labels cannot serve as an independent check on a reference set used to score it.

A larger validation with external cybersecurity professionals was designed but not executed within the project timeframe; the complete survey instrument and analysis plan are provided in Appendix B as a ready-to-run protocol for future work.

3.5 Automated mapping methods

Seven methods are evaluated, chosen to span three families: lexical rule-based baselines, transformer embedding models, and LLM-based approaches.

3.5.1 Rule-based baselines

TF-IDF cosine similarity. Each provision is represented as a TF-IDF-weighted bag of words; every cross-standard pair receives the cosine similarity of the two vectors. This captures shared vocabulary and nothing else, making it the canonical lexical baseline.

Security-keyword Jaccard. A curated list of cybersecurity terms (password, authentication, encryption, vulnerability, logging, risk, update, and others) is matched against each provision; a pair is scored by the Jaccard overlap of the keyword sets. This baseline mimics how a checklist-driven analyst might shortlist related provisions.

3.5.2 Transformer embedding models

Three pretrained encoders embed each provision text into a dense vector; pairs are scored by cosine similarity of the embeddings:

- **Sentence-BERT** (`all-MiniLM-L6-v2`) [10], trained specifically to make cosine similarity meaningful at sentence level;
- **BERT** (`bert-base-uncased`) [9], with mean pooling over token embeddings and L2 normalisation — a general-purpose encoder not tuned for sentence similarity, included to test whether such tuning matters;
- **SecureBERT** [28], a cybersecurity-domain RoBERTa model, embedded the same way, included to test whether domain vocabulary helps.

3.5.3 LLM-based methods

Gemini embeddings. Each provision is embedded with Google’s `gemini-embedding-2` model via API; pairs are scored by cosine similarity, exactly as for the local encoders. This represents the current generation of commercial embedding models.

Gemini LLM classification. For every one of the 4,968 pairs, a prompt containing both provision texts and the six label definitions asks `gemini-2.5-flash-lite` to answer with exactly one label. Jobs are submitted through the batch API. Unlike the similarity methods, this approach predicts a relationship type directly and can, in principle, distinguish overlap from complementarity — something a scalar similarity cannot.

3.5.4 From similarity scores to labels

The similarity-based methods produce scores in $[0, 1]$, not labels. Scores are mapped to relationship types with fixed thresholds: ≥ 0.85 EQUIVALENCE, ≥ 0.70 OVERLAP, ≥ 0.50 COMPLEMENTARITY, otherwise NO RELATION (the rule-based methods use lower entry thresholds reflecting their score distributions). These cut-offs follow the operating points reported in related mapping work [31] and are deliberately kept identical across methods so that differences in results reflect the representations, not per-method tuning. The threshold sensitivity analysis in Section 5.4 examines how much this choice matters. Subsumption cannot be expressed as a symmetric similarity band at all — a structural limitation of similarity-based methods that the results quantify.

3.6 Evaluation framework

Evaluation is restricted to the 107 annotated pairs; predictions outside the annotated set are ignored, since their true labels are unknown. Two views are computed for every method.

3.6.1 Pair detection

Detection asks whether a method finds related pairs at all, regardless of label. A pair is a positive if its ground-truth label is anything but NO RELATION. With TP , FP , FN counted within the annotated scope:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}. \quad (3.1)$$

One consequence of scoping deserves emphasis: false positives can only be counted on the 22 annotated negative pairs. A method that predicts relations sparsely can therefore show precision of 1.0 while missing nearly everything; precision values must be read together with recall and with the predicted-positive counts reported alongside.

3.6.2 Relationship classification

Classification asks whether the predicted label matches the ground truth, over all 107 pairs, with the subsumption directions merged (Section 3.3). Ground-truth pairs absent from a method’s output count as predicted NO RELATION. Reported metrics are overall accuracy and macro-averaged precision, recall, and F1 over the five classes, plus per-class scores. Macro averaging weights all classes equally, which is appropriate here because the rare classes (equivalence, subsumption) are exactly the ones a compliance analyst cares most about.

3.6.3 Uncertainty and sensitivity

With 107 evaluation pairs, point estimates alone would overstate certainty. Nonparametric bootstrap confidence intervals (2,000 resamples of the annotated pairs, percentile method, 95% level) are reported for detection F1 and macro-F1. For every method with a stored similarity matrix, detection metrics are additionally computed across the full threshold range $[0.05, 0.975]$ to separate the quality of the underlying representation from the choice of operating point.

Corpus-level results are reported at two operating points, because the shipped thresholds were selected on the pilot set and applying them to a corpus with a different base

rate confounds the method with its calibration. The second point is chosen on the probability sample itself: weighted F1 is maximised on four folds and scored on the fifth, over 20 repeats, with folds stratified by sampling stratum so each carries all three weight classes. Candidate cut-offs are quantiles of each method's own score distribution rather than a shared absolute grid, since the methods occupy ranges that differ by an order of magnitude in width. The reported threshold is the median of the 100 selections, so the point estimate that follows is not the in-sample optimum; the gap between it and the cross-validated mean is reported as the residual optimism.

Comparisons between methods use a paired bootstrap on the difference: both methods are scored on the same within-stratum resample, and only differences whose interval excludes zero are reported as orderings. Two marginal intervals overlapping is not evidence that the methods are equivalent, because the methods are scored on identical pairs and their errors are correlated.

3.6.4 Efficiency and interpretability

Wall-clock runtime of each stage is recorded on commodity hardware (Apple silicon laptop, CPU only). Interpretability is assessed qualitatively: whether a method's output carries any human-readable rationale (matched keywords, a similarity score, or a generated label) that an analyst could act on.

4 | The Mapping Pipeline

The methods of Chapter 3 are implemented as a five-stage pipeline, released as an open repository so that every number in Chapter 5 can be regenerated from the raw standard documents. This chapter describes the system’s structure, what was built for the project and what was reused, and the artefacts each stage produces.

4.1 Architecture

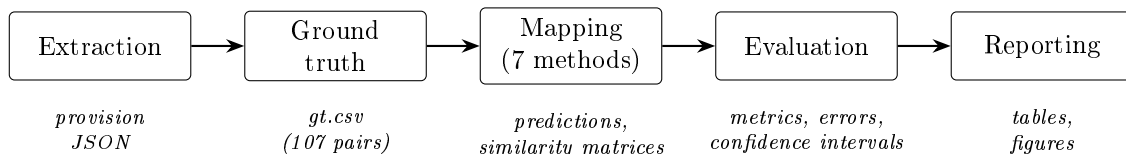


Figure 4.1: Pipeline stages and the artefact each stage produces. Every stage is a separate command, so intermediate outputs can be inspected before the next stage runs.

Figure 4.1 shows the five stages. The pipeline is a set of Python modules run stage by stage from the command line; each stage reads the files of the previous one and writes its results to a versioned data directory. This staged design was a deliberate choice over a single end-to-end script: extraction errors are cheaper to catch before mapping runs, and the API-based methods (which cost money per call) are only invoked once their inputs are verified.

4.2 Stage descriptions

Extraction. Converts both standard PDFs to text with `pdftotext` (Poppler), removes recurring ETSI page furniture, locates provision identifiers with pattern matching, and writes one JSON record per provision with identifier, section, title, text, and modality. Output: 69 records for ETSI EN 303 645, 72 for ETSI EN 304 223.

Ground truth. Materialises the reviewed annotation of Section 3.4 as `gt.csv`: source and target provision identifiers, one of six relationship labels, a one-sentence justification, and a confidence score. A companion module draws the stratified probability sample, emits and ingests the annotation batches, and computes the agreement statistics of Section 3.4.3.

Mapping. One module per method. The rule-based module computes TF-IDF cosine similarity (scikit-learn) and security-keyword Jaccard scores. The three local transformer modules share a common embedding helper that loads the model (Hugging Face), embeds all provisions, normalises the vectors, and computes the full 69×72 cosine similarity matrix. The Gemini embedding module obtains vectors through the Google GenAI API; the Gemini classification module builds one prompt per pair and submits them through the

batch API, then parses the returned labels. Every similarity-based module stores both its thresholded predictions (CSV) and its raw similarity matrix (NumPy), the latter enabling the threshold sweep of Section 5.4 without re-running any model.

Evaluation. Loads the ground truth and every prediction file, computes detection and classification metrics (Section 3.6), bootstrap confidence intervals, per-method confusion matrices, and threshold sweeps, and writes per-method error files listing every missed pair and every wrong label with its provision identifiers.

Reporting. Renders the summary tables and the figures used in Chapter 5 (matplotlib), in print-ready PDF form.

4.3 Built versus reused

The extraction logic, the keyword baseline, the ground-truth tooling, the evaluation harness with its bootstrap and sweep analyses, and the reporting layer were developed for this project. The heavy lifting inside the mapping methods deliberately reuses standard components: scikit-learn’s TF-IDF vectoriser, pretrained models from Hugging Face (MiniLM Sentence-BERT, BERT base, SecureBERT), and Google’s hosted Gemini models. No model was trained or fine-tuned; assembling and evaluating existing components under controlled, identical conditions is precisely the contribution.

4.4 Reproducibility

All randomised steps (sampling, bootstrap) use fixed seeds. A verification run during the project reproduced every stored evaluation metric exactly; individual embedding scores vary in the fourth decimal between runs because of CPU thread scheduling in the tensor library, which never crosses a label threshold. The API-based methods depend on hosted models and are archived through their stored outputs instead. Appendix C lists the exact commands, software versions, and expected runtimes for a full reproduction.

5 | Results and Discussion

This chapter reports the experimental results. Section 5.1 recalls the setup, Sections 5.2–5.6 present pair detection, similarity structure, threshold sensitivity, and relationship classification, Section 5.7 covers the ground-truth validation study, and Sections 5.8–5.10 discuss the findings against the research questions and the threats to their validity.

5.1 Experimental setup

All seven methods were run on the full cross product of 69 ETSI EN 303 645 provisions and 72 ETSI EN 304 223 provisions (4,968 pairs) and evaluated on the 107 annotated pairs: 85 related pairs and 22 annotated negatives (Figure 5.1). The distribution is dominated by OVERLAP (51 pairs), with EQUIVALENCE rare (3 pairs) — the two standards describe different system layers, so identical requirements barely occur. Metrics outside the annotated scope are not computable and are not claimed.

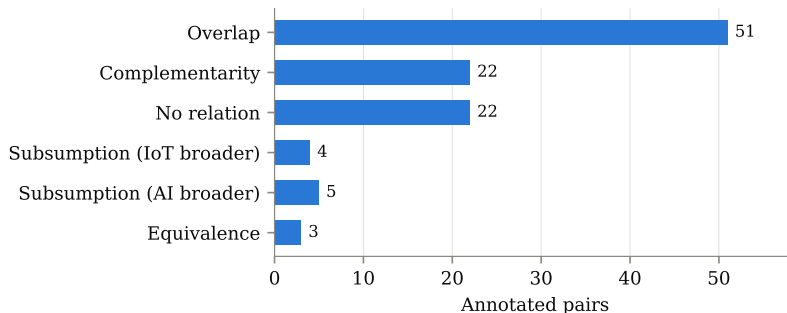


Figure 5.1: Distribution of relationship labels in the 107-pair ground truth.

5.2 Pair detection

Table 5.1 and Figure 5.2 show the detection results; Figure 5.3 adds bootstrap confidence intervals.

The table splits the methods into two regimes. The lexical baselines and SentenceBERT at its default operating point predict very few relations; what they do predict is correct (no false positives), but recall between 0.02 and 0.17 makes them useless as stand-alone detectors. The remaining methods sit at the opposite extreme. SecureBERT and Gemini embeddings mark *every* annotated pair as related, and BERT marks all but two. Their high F1 (≈ 0.89) must therefore be read with care: with 85 of 107 annotated pairs being positives, the trivial strategy “call everything related” already achieves $F1 = 0.885$. BERT, SecureBERT, and Gemini embeddings essentially implement that strategy, and the Gemini LLM ($F1 = 0.816$) stays below it. Detection F1 within the annotated scope, in other words, barely distinguishes these methods; the informative differences appear in the similarity structure and in classification.

Table 5.1: Pair detection on the annotated scope. P^+ is the number of pairs the method marks as related among the 107 annotated pairs. Precision counts false positives only on the 22 annotated negatives, which is why sparse methods show precision 1.00 despite very low recall.

Method	P^+	TP	FP	FN	Precision	Recall	F1
TF-IDF	3	3	0	82	1.000	0.035	0.068
Keyword Jaccard	6	6	0	79	1.000	0.071	0.132
Sentence-BERT	15	15	0	70	1.000	0.176	0.300
BERT	105	85	20	0	0.809	1.000	0.895
SecureBERT	107	85	22	0	0.794	1.000	0.885
Gemini embeddings	107	85	22	0	0.794	1.000	0.885
Gemini LLM	94	73	21	12	0.777	0.859	0.816

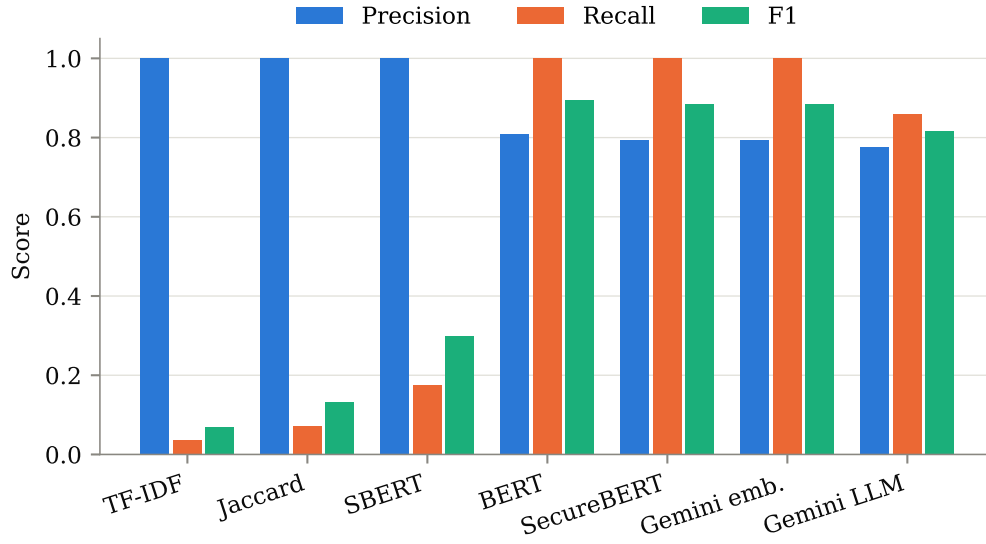


Figure 5.2: Pair-detection precision, recall, and F1 per method.

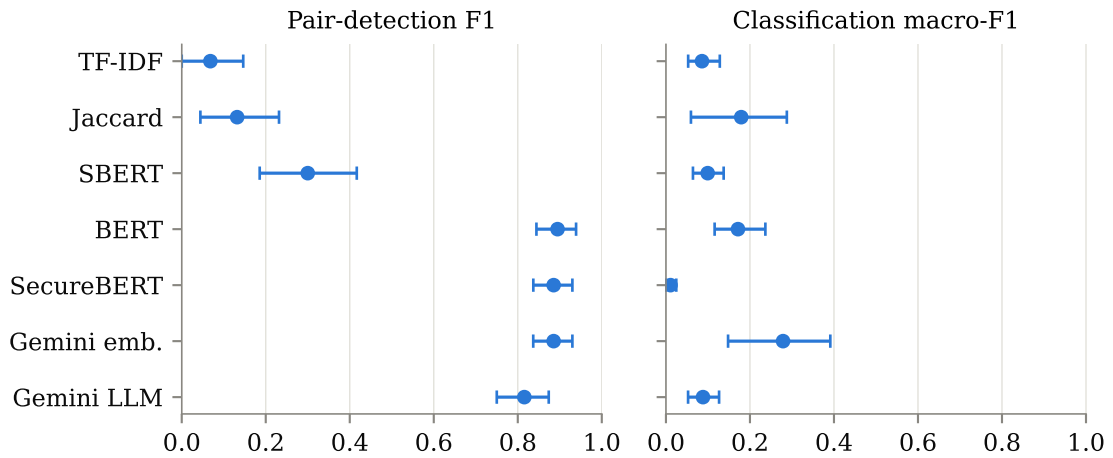


Figure 5.3: Bootstrap 95% confidence intervals (2,000 resamples) for pair-detection F1 and classification macro-F1.

5.3 Similarity structure

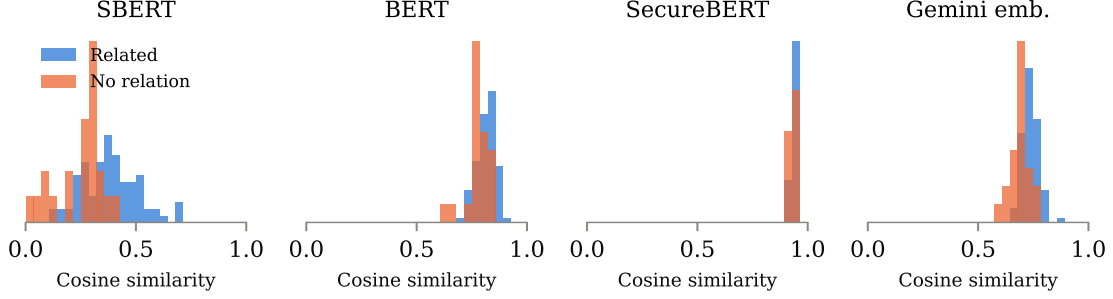


Figure 5.4: Distributions of cosine similarity for annotated related pairs (blue) and annotated negatives (orange), per embedding model.

Figure 5.4 explains the degenerate behaviour. BERT concentrates all pairs — related or not — between roughly 0.6 and 0.9; SecureBERT is even more extreme, packing everything above 0.9. With such compressed score ranges, the fixed label thresholds of Section 3.5.4 land every pair in the same band: SecureBERT assigns EQUIVALENCE to all 4,968 possible pairs. This matches the known anisotropy of contextual embedding spaces [29]: mean-pooled vectors from models never trained for sentence similarity crowd into a narrow cone, and cosine similarity loses its discriminative meaning. Sentence-BERT, trained precisely to spread sentence meanings out, shows the widest score range and visibly separates the two distributions — yet even there the overlap is substantial, confirming that annotated negatives (chosen to share surface vocabulary) are genuinely hard. Gemini embeddings sit between the extremes: compressed range, but with related pairs shifted consistently above negatives.

5.4 Threshold sensitivity

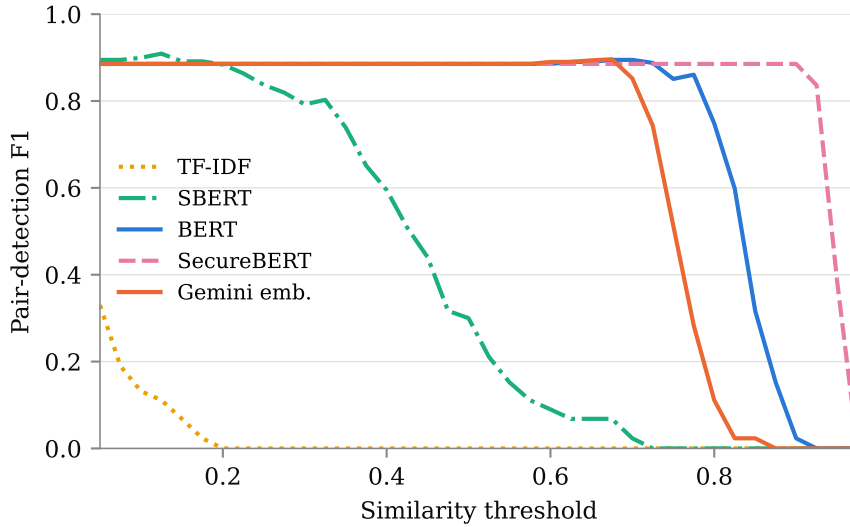


Figure 5.5: Pair-detection F1 as a function of the similarity threshold, computed from the stored similarity matrices on the annotated scope.

Figure 5.5 separates representation quality from the choice of operating point. Three observations stand out. First, the plateaus of BERT, SecureBERT, and Gemini embeddings coincide with the all-positive baseline ($F1 \approx 0.885$) across most of the threshold range — confirming that their default-threshold results reflect saturation, not discrimination. Second, with an optimally chosen threshold Sentence-BERT becomes the *best* detector ($F1 = 0.909$ at $t = 0.120$), despite being the worst non-trivial one at its default operating point; its published default entry threshold is simply far too high for cross-standard requirement text. Choosing a threshold on the same pairs the score is reported on inflates it, so the sweep was repeated under 5-fold cross-validation with 20 repetitions: the threshold is selected on four folds and scored on the fifth. Held-out F1 is 0.903 ± 0.021 , an optimism of only 0.006, because a single tuned parameter over 107 points has little room to overfit. Figure 5.6 shows the same comparison for every method with a continuous score, together with the spread of the selected threshold itself. The two panels carry different news: held-out F1 tracks the in-sample optimum closely everywhere, but the threshold that produces it is stable only for some models. Gemini embeddings settle on 0.658 ± 0.023 across all 100 fits, whereas SecureBERT ranges over 0.265 ± 0.375 — its folds disagree about the operating point by more than the width of the useful range, which is the same brittleness the sweep curve shows from a different angle. Calibration gain is therefore real, though Section 5.5 shows it is measured at a positive rate the corpus does not share. Third, the sharp cliffs on the right show how brittle threshold-based labelling is: for SecureBERT the entire working range collapses within a few hundredths. For practice this means thresholds must be calibrated per corpus and per model — fixed literature values do not transfer.

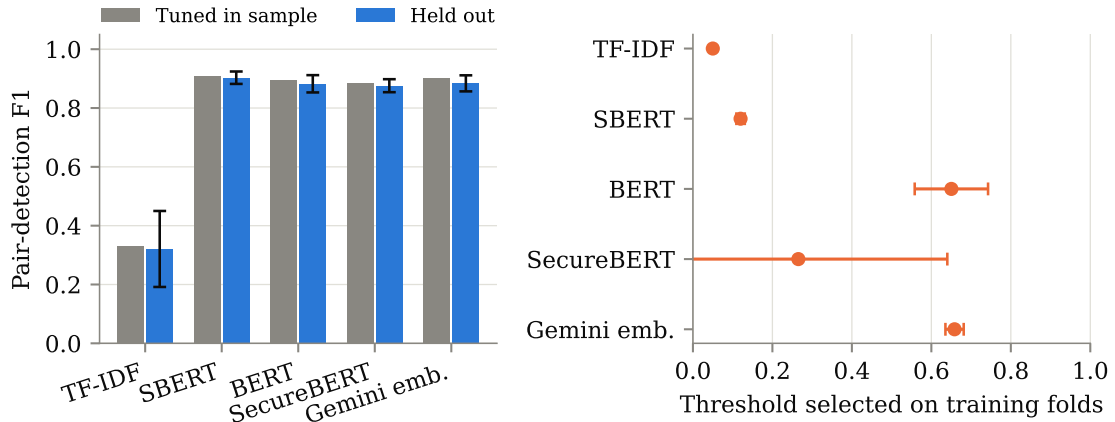


Figure 5.6: Left: pair-detection F1 with the threshold tuned on the same pairs it is scored on, against the held-out mean of 20 repeats of stratified 5-fold cross-validation (error bars, one standard deviation). Right: the threshold each set of training folds selected.

5.5 Corpus-level performance

Every figure reported so far describes the 107 pilot pairs, of which 85 are related — a positive rate of 79.4%. That set was assembled purposively, so it carries no inclusion probabilities and its precision cannot be reweighted to the corpus it was drawn from. Because precision and F1 both move with the positive rate, this matters: a method evaluated where four pairs in five are related is being asked an easier question than the one it would face in use.

To measure rather than assume the difference, a stratified probability sample of the full 69×72 space was drawn and annotated independently (Section 3.4.2). Each pair carries a known inclusion probability, so Horvitz–Thompson estimation recovers corpus-level quantities with correct variance.

How common are related pairs? The sample puts the corpus rate at **12.4%** (95% CI [10.0, 15.0]) for any relationship, and **5.9%** (95% CI [4.3, 7.7]) counting only equivalence, overlap and subsumption. Applied to 4,968 candidates that implies roughly 617 and 295 related pairs respectively. The pilot’s 85 was never a measurement of this quantity — it was the number of pairs that study examined.

The unequal sampling fractions cost some precision of their own. With weights ranging from 1.99 to 14.91, Kish’s design effect is 1.49, so the 650 annotated pairs carry the information of roughly 437 drawn with equal probability. Every interval reported below already accounts for this, because the bootstrap resamples within stratum.

Table 5.2: Detection performance estimated over all 4,968 candidate pairs from the weighted sample, at each method’s shipped operating point. Specificity is the fraction of unrelated pairs correctly rejected. The final row is a classifier that labels every pair related.

Method	Precision	Recall	Specificity	F1
Sentence-BERT	0.905	0.061	0.999	0.115
Keyword Jaccard	0.101	0.019	0.976	0.033
TF-IDF	0.000	0.000	1.000	0.000
BERT	0.128	1.000	0.038	0.228
Gemini LLM	0.121	0.886	0.092	0.214
SecureBERT	0.124	1.000	0.000	0.221
Gemini embeddings	0.124	1.000	0.000	0.221
<i>Label every pair related</i>	<i>0.124</i>	<i>1.000</i>	<i>0.000</i>	<i>0.221</i>

The ranking inverts. Table 5.2 does not rearrange the pilot ordering so much as contradict it. SecureBERT and Gemini embeddings reach specificity 0.000: across the weighted sample they reject no unrelated pair at all, and their scores match the all-positive baseline to three decimals. BERT, which led the pilot benchmark at detection F1 0.895, rejects 3.8% of unrelated pairs. A paired bootstrap on the difference in F1 — both methods scored on the same resampled pairs, which is the comparison the overlap of two marginal intervals does not make — puts SecureBERT, Gemini embeddings and the Gemini LLM within the baseline’s interval, and BERT ahead of it by 0.007 [0.003, 0.011], a margin with no practical content.

Sentence-BERT moves the other way. It looked worst among the non-degenerate methods on the pilot set (F1 0.300) and is the only method here with precision of practical interest: 0.905, meaning nine in ten of the pairs it surfaces are genuinely related. It pays for this with recall of 0.061.

Sensitivity and specificity are properties of a classifier alone; precision also depends on how common the positive class is. Fixing the first two therefore fixes precision as a function of prevalence, plotted in Figure 5.7. Three of the four methods shown lie on the diagonal precision = prevalence, the locus of a classifier that carries no information about the label: at the pilot’s 0.79 they return 0.79, which reads as strong performance, and at the corpus rate of 0.12 they return 0.12. Only Sentence-BERT’s curve stands away from the diagonal, and it is the separation between curve and diagonal — not the height of either — that

measures discrimination.

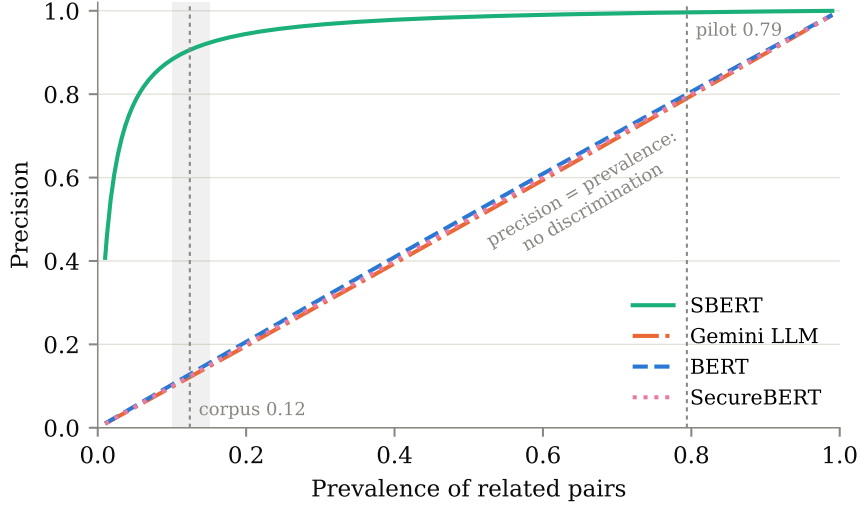


Figure 5.7: Precision implied by each method’s estimated sensitivity and specificity as the share of related pairs varies. Dotted lines mark the pilot set’s positive rate and the corpus estimate; the shaded band is the 95% interval on the latter.

How much of the collapse is the threshold? Table 5.2 scores each method at the operating point Chapter 4 shipped, and those thresholds were chosen on the pilot set. Selecting a cut-off where 79% of pairs are related and then applying it where 12% are confounds the method with its calibration, so the comparison was repeated with the operating point selected on the probability sample itself: weighted F1 maximised on four folds and scored on the fifth, 20 repeats, stratified so every fold carries all three strata, with the reported threshold the median of the 100 selections rather than the in-sample optimum. Candidate cut-offs are quantiles of each method’s own scores, since the methods occupy very different ranges — TF-IDF tops out at 0.14 while SecureBERT never drops below 0.91, and a shared absolute grid would give one of them a hundred candidate thresholds and the other three.

Table 5.3: Detection performance when the threshold is selected on the probability sample instead of the pilot set. Held-out F1 is the cross-validated mean; the gap to the point estimate is the remaining optimism.

Method	Threshold	Precision	Recall	F1	Held-out F1
Gemini embeddings	0.719	0.446	0.571	0.501	0.485
Sentence-BERT	0.324	0.327	0.571	0.416	0.378
TF-IDF	0.017	0.281	0.367	0.319	0.285
BERT	0.824	0.233	0.477	0.313	0.289
SecureBERT	0.939	0.178	0.508	0.264	0.252
<i>Label every pair related</i>	—	<i>0.124</i>	<i>1.000</i>	<i>0.221</i>	—

The result changes the diagnosis (Figure 5.8). At a threshold fitted to the corpus, four of the five continuous methods clear the all-positive baseline, and two clear it by a margin the paired bootstrap separates from zero: Gemini embeddings by 0.280 [0.199, 0.365] and Sentence-BERT by 0.195 [0.123, 0.263]. TF-IDF and BERT beat it by about 0.09 each,

intervals excluding zero but only just. SecureBERT does not clear it at all ($+0.043$, $[-0.011, 0.096]$). The two leaders are not separable from one another (-0.085 , $[-0.184, 0.013]$), so the honest statement is that they are tied at the top, not that one wins.

What collapsed in Table 5.2, then, was mostly calibration rather than representation. The embedding spaces do carry enough signal to reject unrelated pairs; the thresholds inherited from a 79%-positive development set did not let them. That is a sharper version of the same warning: the cost of a development set at the wrong base rate is not a mild optimism in the reported score, it is an operating point that turns a usable model into a constant function. SecureBERT is the exception, and the informative one — its scores occupy a band 0.06 wide across the entire corpus, so no threshold placed anywhere in that band separates the classes.

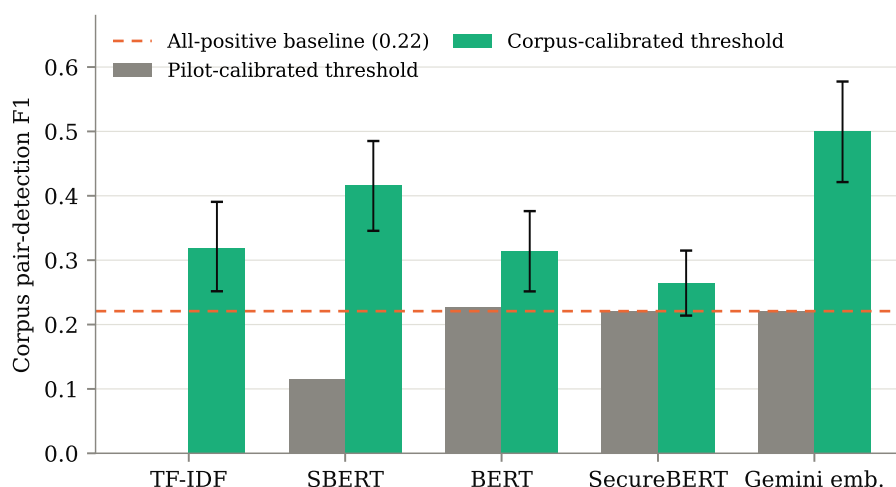


Figure 5.8: Corpus pair-detection F1 at the shipped, pilot-calibrated threshold against a threshold cross-validated on the probability sample. Error bars are stratified bootstrap 95% intervals.

What this supports. Even recalibrated, no method here supports an automated accept/reject decision: the best F1 is 0.50, with precision 0.45, so more than half of what the system proposes is wrong. The defensible use is the one Section 5.9 describes — ranking candidates for a human reviewer — and the choice of configuration depends on which end of the trade-off matters. Sentence-BERT at its shipped threshold surfaces few pairs but is right about nine times in ten; Gemini embeddings recalibrated recover 57% of the related pairs at under half precision. Neither removes the reviewer.

5.6 Relationship classification

Detecting that two provisions relate is only half the task; compliance work needs to know *how* they relate. Table 5.4 summarises classification performance.

Classification is where every method struggles. The best result — Gemini embeddings at 0.477 accuracy and 0.279 macro-F1 — would still mislabel every second pair. The comparison that matters is against the trivial classifier that assigns the majority label to everything: OVERLAP accounts for 51 of the 107 pairs, so always predicting it yields 0.477 accuracy. The best method matches that figure exactly and none exceeds it. The same baseline scores 0.129 macro-F1, which three methods do beat — keyword Jaccard, BERT and Gemini embeddings — so the methods are not uniformly worthless; but the headline

Table 5.4: Relationship classification over all 107 annotated pairs, five-label scheme (subsumption directions merged). CIs are bootstrap 95% intervals.

Method	Accuracy	Macro-F1	Macro-F1 95% CI
TF-IDF	0.215	0.086	[0.053, 0.128]
Keyword Jaccard	0.224	0.179	[0.059, 0.288]
Sentence-BERT	0.224	0.099	[0.064, 0.137]
BERT	0.411	0.172	[0.116, 0.237]
SecureBERT	0.028	0.011	[0.000, 0.025]
Gemini embeddings	0.477	0.279	[0.148, 0.391]
Gemini LLM	0.187	0.088	[0.053, 0.127]

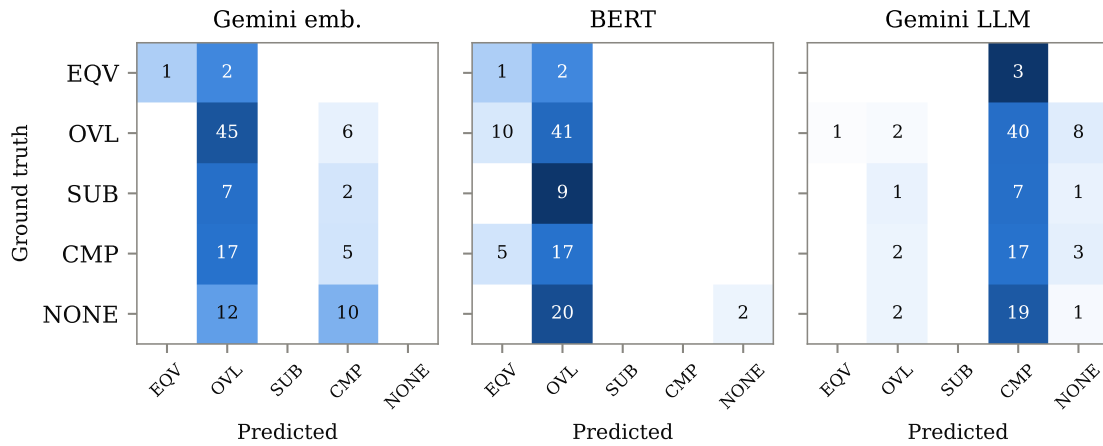


Figure 5.9: Row-normalised confusion matrices (counts printed) for the three most instructive methods. EQV = equivalence, OVL = overlap, SUB = subsumption, CMP = complementarity, NONE = no relation.

conclusion has to be stated plainly. On the five-label task, none of the seven methods learns the distinction the task is about. The confusion matrices in Figure 5.9 show how each one fails:

- **Gemini embeddings** funnel most predictions into OVERLAP. That matches the majority class (hence the comparatively high accuracy) and yields reasonable per-class scores for OVERLAP (F1 = 0.65) and EQUIVALENCE (precision 1.00 at recall 0.33), but SUBSUMPTION and NO RELATION are never predicted correctly.
- **BERT** spreads slightly more but shows the same collapse; COMPLEMENTARITY and SUBSUMPTION both go to zero.
- **The Gemini LLM** is the only method that predicts labels from provision content rather than a score band, and it does recognise complementarity (recall 0.59) — but it heavily overuses it, pushing 41 of the 51 OVERLAP pairs into COMPLEMENTARITY and ending with the lowest accuracy of the non-degenerate methods. Notably, it also assigns a relation to 20 of the 22 annotated negatives.
- **SecureBERT**’s uniform EQUIVALENCE output produces 0.028 accuracy — exactly the 3 equivalence pairs in the ground truth.

Two error examples illustrate the pattern. Provision 5.3-11 of ETSI EN 303 645 (inform the user that a security update is required, with its risks) and provision 5.3.1-2.2 of ETSI EN 304 223 (proactively inform end-users of security-relevant updates with clear explanations) are annotated EQUIVALENCE; Gemini embeddings scored the pair into the OVERLAP band — a near-miss that a human reviewer would accept as a shortlist hit. By contrast, the Gemini LLM labelled the pair of provision 5.1-1 (unique per-device passwords) and provision 5.2.2-1 (evaluate access control frameworks for APIs, models, and data) as COMPLEMENTARITY where the annotation says OVERLAP: both provisions govern authentication-related access control, but the model read the difference in system layer as a difference in objective. The boundary between overlap and complementarity is exactly where the annotation itself is hardest, a point the validation study below quantifies.

5.7 Ground-truth validation

A reference set is only as good as the consistency of the judgements behind it. The pilot pairs were mixed unmarked into the annotation stream for the probability sample, so relabelling them was blind: at the time of judging, there was no way to tell a pilot pair from a newly sampled one. Comparing the two label sets afterwards gives an agreement figure against annotations that two authors reviewed months earlier.

Table 5.5: Agreement of the annotated reference sample with the earlier author-reviewed pilot labels, against the same statistic computed for the evaluated LLM on two runs over identical input.

Comparison	n	Six labels		Related vs. not	
		Agreement	κ	Agreement	κ
Reference sample vs. author-reviewed pilot	107	0.61	0.40	0.82	0.51
Evaluated LLM vs. itself, identical input	4 968	0.69	0.02	0.81	−0.03

Agreement with the pilot reaches $\kappa = 0.51$ on the detection decision and $\kappa = 0.40$ on the six-label scheme — moderate on both counts on the Landis–Koch scale [33]. This is

weaker than it looked partway through annotation, when the 47 pilot pairs reached so far gave 0.68 and 0.44; the figures above are over all 107 and are the ones that should be read. A reference set at $\kappa = 0.51$ on its central decision is usable for comparing methods against each other, since every method is scored against the same labels, but it is not precise enough to certify an absolute performance level, and the method rankings in Section 5.5 should be read with that ceiling in mind.

Disagreements are not scattered across the label space. They cluster on the overlap/complementarity boundary, which Section 5.10 identifies as the least determinate distinction in the taxonomy, and on the general-process provisions covered by the codebook amendment recorded during annotation. Recorded confidence tracks them: pairs marked confidence 3 agree with the pilot 77% of the time against 46% for confidence 2. The scale collapsed in practice to those two values — no pair was recorded at 1, 4 or 5 — so this is a two-level contrast rather than a calibration curve, but it does show the field carries information rather than noise.

The second row of Table 5.5 is the more instructive one. The evaluated LLM was run twice at temperature 0 over byte-identical prompts. The two runs agree on 69% of pairs — *higher* raw agreement than the human comparison — yet reach $\kappa = 0.02$. The explanation is that the model assigns COMPLEMENTARITY to 81% of all pairs, so two runs coincide frequently without either run carrying information. Raw agreement rewards a degenerate marginal; κ does not. This is why the reliability of an annotation source cannot be read off its agreement percentage, and it is the clearest single demonstration in this study of why the LLM classifier’s headline scores should not be taken at face value.

Recorded annotation confidence behaves as intended: pairs marked confidence 3 agree with the pilot 82% of the time ($n = 22$), against 48% for those marked confidence 2 ($n = 25$). The field predicts where judgement is contestable, so it is retained rather than discarded.

5.8 Efficiency and interpretability

Table 5.6: Wall-clock runtime per stage on a CPU-only laptop (Apple silicon), including model loading. Gemini stages run through the batch API; their end-to-end time is dominated by queueing (minutes to hours) rather than computation.

Stage	Runtime
Provision extraction (both PDFs)	0.9 s
TF-IDF + keyword mapping	3.5 s
Sentence-BERT mapping	56 s
BERT mapping	104 s
SecureBERT mapping	195 s
Gemini embeddings	API batch
Gemini LLM classification (4,968 prompts)	API batch
Evaluation	1.4 s

Every local method runs in minutes on commodity hardware (Table 5.6); by comparison, drafting and reviewing the 107 pilot pairs occupied several working days even with LLM assistance, and the 650 sampled pairs took several times that again. Interpretability differs more than runtime: the keyword baseline can show *which* terms matched, embedding methods offer only an opaque score, and the LLM can produce a textual rationale on request — though its rationale quality was not separately evaluated here.

5.9 Discussion

RQ1 — feasibility and limits. Automated methods can reliably *shortlist* related provisions: the better embedding models place genuinely related pairs high in their rankings, and with a calibrated threshold Sentence-BERT reaches detection F1 above 0.9 on the annotated scope — although, as Section 5.5 shows, that figure reflects the benchmark’s high positive rate more than deployable accuracy. Identifying the *type* of relationship is a different matter. The best classification result (macro-F1 0.279) is far from usable autonomy, and structural limits are visible: symmetric similarity scores cannot express directional subsumption at all, and the overlap/complementarity boundary requires judging *objectives*, not surface meaning. The fundamental limitation is that a scalar similarity collapses precisely the distinctions the taxonomy is designed to capture.

RQ2 — method comparison. No method family wins outright; they fail differently. Lexical baselines are precise but nearly blind (recall ≤ 0.07). General-purpose transformer embeddings saturate — anisotropy makes their cosine scores unusable without calibration, and domain pretraining (SecureBERT) makes this worse rather than better, refuting the intuition that cybersecurity vocabulary is what is missing. The purpose-built sentence encoder (Sentence-BERT) has the best underlying representation once its threshold is calibrated. Gemini embeddings give the best label quality overall (macro-F1 0.279, accuracy 0.477), and the prompted LLM is the only method with non-trivial recall on complementarity but over-predicts it massively. The pilot and corpus views disagree about which method to prefer, and where they disagree the corpus view should win: Gemini embeddings and Sentence-BERT lead once the operating point is fitted to the corpus rather than to the pilot, and the paired bootstrap cannot separate them (-0.085 , $[-0.184, 0.013]$). Since accuracy does not decide between the two, the practical grounds do: Sentence-BERT runs locally in under a minute with no per-query cost and reproduces bit-identically, whereas the Gemini embeddings depend on a hosted model that can change under the same name. Calibrated Sentence-BERT for ranking, plus human labelling of the shortlist, is the defensible configuration, with Gemini embeddings the better choice where recall matters more than reproducibility.

RQ3 — automated versus human mapping. Against the author-reviewed reference, automation is dramatically faster (seconds versus days) and consistent, but its label agreement (at best 46%) remains far below what compliance documentation requires. These findings support the same conclusion Bar-Haim et al. [13] reached for control mapping: automated compliance mapping is a decision-support technology. Concretely, the evidence here supports a workflow where an embedding model ranks all 4,968 pairs, a human reviews the top of the ranking, and the relationship label is always assigned by the human.

5.10 Threats to validity

Construct validity. The six-label taxonomy compresses a continuum of relationships between requirements into discrete classes; the overlap/complementarity boundary in particular is partly a judgement call. Section 5.7 puts a number on the cost: agreement on the binary related/unrelated question reaches $\kappa = 0.51$, and resolving *which* of the six relations holds falls to $\kappa = 0.40$, so the typed labels are the noisier construct and the classification results in Section 5.6 are measured against a correspondingly noisier target.

Internal validity. Both reference sets were drafted with LLM assistance, which risks favouring LLM-family methods. Three things bound the risk. The drafting model (Claude) belongs to a different family from the evaluated ones (Gemini); the pilot’s 107 pairs were reviewed by the authors; and the evaluated LLM performed *worst* of the non-degenerate methods, so the circularity, if any, did not manifest as an advantage. The probability sample was not author-reviewed pair by pair, and its reliability rests instead on the blind comparison with the reviewed pilot reported in Section 5.7. Fixed shared thresholds avoid per-method tuning bias; the sweep and the cross-validated calibration show what tuning would and would not change.

Statistical conclusion validity. Three limits constrain how far the numbers can be pushed. First, the pilot-set intervals in Figure 5.3 overlap widely across the stronger methods; with 107 pairs, and with EQUIVALENCE supported by only three of them, the ranking between neighbouring methods on that set is not established. Corpus-scale comparisons are made instead with a paired bootstrap on the difference, which is the test that matches the design, and only differences whose interval excludes zero are reported as orderings. Second, EQUIVALENCE does not occur in the probability sample at all, so no weighted per-class estimate exists for it and the classification results remain confined to the pilot set. Third, the corpus operating points are selected by cross-validation on the same sample they are scored on. The residual optimism is measured rather than assumed — 0.016 for Gemini embeddings, 0.048 for Sentence-BERT — and is small relative to the gap between those methods and the baseline, but the point estimates in Table 5.3 sit slightly above what a fully independent test set would give.

External validity. Results cover one standard pair and specific model versions; hosted models additionally evolve over time, and `gemini-2.5-flash-lite` is scheduled for retirement in October 2026, after which the LLM results here cannot be reproduced exactly. Generalisation to other standards or model generations is plausible for the structural findings (anisotropy, similarity-versus-taxonomy mismatch) but must be assumed with caution for the specific numbers. The prevalence gap between the two reference sets is itself the clearest warning: a benchmark assembled by looking for interesting pairs measured something the deployed setting does not share.

6 | Conclusion

This thesis asked how far the mapping of security requirements between ETSI EN 303 645 and ETSI EN 304 223 can be automated with current NLP and LLM techniques. A five-stage pipeline was built that extracts 141 normative provisions from the two standards, maps them with seven methods spanning lexical, embedding, and LLM approaches, and evaluates all of them against two reference sets: an author-reviewed pilot of 107 pairs and a stratified probability sample of 650 pairs drawn from the full candidate space.

The answer the evidence supports is a split one, and it depends on where the question is asked. On the 107-pair pilot set, finding related provisions looks largely feasible: with a calibrated threshold the best model reaches pair-detection F1 above 0.9, and cross-validation confirms that figure is not an artefact of tuning. Estimated over the whole candidate space from the probability sample, the picture changes. Related pairs make up 12.4% of the space rather than the pilot’s 79%, and at that base rate four of the seven methods score what a classifier that labels every pair related would score.

Most of that collapse turns out to be calibration rather than capability. The thresholds were selected on the pilot set, and re-selecting them by cross-validation on the probability sample lifts Gemini embeddings to F1 0.501 and Sentence-BERT to 0.416 against an all-positive baseline of 0.221, margins that a paired bootstrap separates from zero. The two leaders are not separable from each other. SecureBERT does not clear the baseline under any threshold, because its scores span a band 0.06 wide across the entire corpus. The practical lesson is sharper than a warning about optimistic benchmarks: a development set at the wrong base rate does not merely inflate the reported score, it fixes an operating point that turns a working model into a constant function. Determining *how* two provisions relate is not solved: the best method (Gemini embeddings) reaches 47.7% label accuracy and 0.279 macro-F1, only matching the accuracy of simply labelling every pair with the majority class, and no similarity-based method can represent directional subsumption even in principle. Two failure mechanisms recur across the experiments — the anisotropy of general-purpose embedding spaces, which drives BERT-family models into degenerate “everything is similar” behaviour, and the mismatch between scalar similarity and a typed relationship taxonomy. Notably, cybersecurity-specific pretraining (SecureBERT) did not help; task-specific training for sentence comparison (Sentence-BERT) did.

For practice, the results position automated compliance mapping as decision support: rank candidates automatically, let humans assign the relationship labels. For a compliance team, even that reduced role is valuable — the ranking step compresses 4,968 candidate pairs into a short review list in under a minute of computation.

Critical discussion

Three choices shaped this project and deserve scrutiny. First, both reference sets were annotated with LLM assistance; this was a pragmatic necessity at 15 credits, it is disclosed openly, and the design (a different model family from the evaluated ones, a written codebook fixed in advance, and a blind comparison against the author-reviewed pilot) bounds the risk. What it does not remove is the ceiling that comparison reveals: $\kappa = 0.51$ on

the detection decision means the reference is good enough to rank methods against a common target but not to certify an absolute accuracy. A fully expert-built reference would be stronger, and the survey in Appendix B is the direct path to one. Second, the typed classification results rest on 107 pairs, because EQUIVALENCE never appears in a 650-pair probability sample and the class cannot be estimated at corpus scale at all. Third, reporting two operating points per method rather than one doubles the tables but is the only way to separate what a model can represent from where its threshold happens to sit.

Generalisation of the results

The specific numbers belong to this standard pair and these model versions. The structural findings travel further: anisotropy of mean-pooled contextual embeddings is a documented model property, not a dataset accident; the inability of symmetric scores to express asymmetric relationships holds for any standard pair; and the observation that detection saturates while typed classification fails should recur wherever two related security frameworks are mapped. The pipeline itself is standard-agnostic — any pair of documents with identifiable normative clauses can be substituted.

Future work

Four directions follow naturally. First, execute the expert validation survey designed in Appendix B to put the ground truth on a multi-rater footing and to calibrate the human agreement ceiling. Second, replace fixed thresholds with learned classifiers over embedding pairs (concatenation or cross-encoders), which could recover subsumption direction from asymmetric features. Third, revisit the prompt given to the LLM classifier. Its overuse of complementarity tracks the wording of the label definition rather than the difficulty of the pairs — “related but different concerns that work together” admits almost any two security requirements — which suggests the ceiling measured here is the definition’s, not the model’s. Fourth, extend the probability sample toward the rare classes. The realised design of 650 pairs pins prevalence to ± 2.5 percentage points but contains no EQUIVALENCE pair, so a targeted supplement — usable for per-class recall and confusion, but excluded from every prevalence-sensitive estimate — is what the typed task needs next. Beyond that, ranking metrics such as precision at k and mean average precision would decouple the evaluation from prevalence altogether, since they measure the shortlisting behaviour the pipeline is actually fit for; the machinery is implemented but the judged fraction of each ranking is still so thin that precision among judged candidates is 1.000 for every method, meaning the numbers describe judging coverage rather than ranking quality.

Bibliography

- [1] M. M. Aslam, A. Tufail, H. Gul, *et al.*, “Artificial intelligence for secure and sustainable industrial control systems – a survey of challenges and solutions,” *Artificial Intelligence Review*, vol. 58, p. 349, 2025.
- [2] B. Alotaibi, “A survey on industrial Internet of Things security: Requirements, attacks, AI-based solutions, and edge computing opportunities,” *Sensors*, vol. 23, no. 17, p. 7470, 2023.
- [3] H. Meziane and N. Ouerdi, “A survey on performance evaluation of artificial intelligence algorithms for improving IoT security systems,” *Scientific Reports*, vol. 13, p. 21255, 2023.
- [4] ETSI, “ETSI EN 303 645 V3.1.3: Cyber security for consumer internet of things: Baseline requirements.” European Telecommunications Standards Institute, 2024.
- [5] ETSI, “ETSI EN 304 223 V2.1.1: Securing artificial intelligence (SAI); baseline cyber security requirements.” European Telecommunications Standards Institute, 2025.
- [6] O. Kosenkov, P. Elahidoost, T. Gorschek, J. Fischbach, D. Mendez, M. Unterkalmsteiner, D. Fucci, and R. Mohanani, “Systematic mapping study on requirements engineering for regulatory compliance of software systems,” *Information and Software Technology*, vol. 178, p. 107622, 2025.
- [7] F. Djebbar and K. Nordström, “A comparative analysis of industrial cybersecurity standards,” *IEEE Access*, vol. 11, pp. 85315–85332, 2023.
- [8] D. Canavese, A. Ferreira, L. Kamm, and A. Q. Rodriguez, “Uncovering challenges of cybersecurity cross-regulation in EU legislation,” *Information and Software Technology*, vol. 193, p. 108033, 2026.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1*, (Minneapolis, Minnesota), pp. 4171–4186, Association for Computational Linguistics, 2019.
- [10] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, (Hong Kong, China), pp. 3982–3992, Association for Computational Linguistics, 2019.
- [11] D. Rodriguez, I. Yang, J. M. Del Alamo, and N. Sadeh, “Large language models: a new approach for privacy policy analysis at scale,” *Computing*, vol. 106, no. 12, pp. 3879–3903, 2024.

- [12] O. A. Cejas, M. I. Azeem, S. Abualhaija, and L. C. Briand, “NLP-based automated compliance checking of data processing agreements against GDPR,” *IEEE Transactions on Software Engineering*, vol. 49, no. 9, pp. 4282–4303, 2023.
- [13] R. Bar-Haim, L. Eden, Y. Kantor, V. Agarwal, M. Devereux, N. Gupta, A. Kumar, M. Orbach, and M. Zan, “Towards automated assessment of organizational cybersecurity posture in cloud,” in *Proceedings of the 6th Joint International Conference on Data Science & Management of Data (CODS-COMAD '23)*, (New York, NY, USA), pp. 167–175, Association for Computing Machinery, 2023.
- [14] S. Sicari, A. Rizzardi, L. Grieco, and A. Coen-Porisini, “Security, privacy and trust in Internet of Things: The road ahead,” *Computer Networks*, vol. 76, pp. 146–164, 2015.
- [15] G. Vardakis, G. Hatzivasilis, E. Koutsaki, and N. Papadakis, “Review of smart-home security using the Internet of Things,” *Electronics*, vol. 13, no. 16, 2024.
- [16] D. Mocrii, Y. Chen, and P. Musilek, “IoT-based smart homes: A review of system architecture, software, communications, privacy and security,” *Internet of Things*, vol. 1–2, pp. 81–98, 2018.
- [17] H. Chen and M. A. Babar, “Security for machine learning-based software systems: A survey of threats, practices, and challenges,” *ACM Computing Surveys*, vol. 56, 2024.
- [18] X. He, G. Xu, X. Han, Q. Wang, L. Zhao, C. Shen, C. Lin, Z. Zhao, Q. Li, L. Yang, *et al.*, “Artificial intelligence security and privacy: a survey,” *Science China Information Sciences*, vol. 68, no. 8, p. 181101, 2025.
- [19] A. Mussmann, M. Brunner, and R. Breu, *Mapping the State of Security Standards Mappings*, pp. 1309–1324. 2020.
- [20] C. Negri-Ribalta, M. Lombard-Platet, and C. Salinesi, “Understanding the GDPR from a requirements engineering perspective — a systematic mapping study on regulatory data protection requirements,” *Requirements Engineering*, vol. 29, no. 4, pp. 523–549, 2024.
- [21] J. P. Castellanos Ardila, B. Gallina, and F. Ul Muram, “Compliance checking of software processes: A systematic literature review,” *Journal of Software: Evolution and Process*, vol. 34, no. 5, p. e2440, 2022.
- [22] A. Sleimi, N. Sannier, M. Sabetzadeh, L. Briand, M. Ceci, and J. Dann, “An automated framework for the extraction of semantic legal metadata from legal texts,” *Empirical Software Engineering*, vol. 26, 2021.
- [23] A. Mahmoud and N. Niu, “On the role of semantics in automated requirements tracing,” *Requirements Engineering*, vol. 20, no. 3, pp. 281–300, 2015.
- [24] D. Rajpathak, P. M. Peranandam, and S. Ramesh, “Automatic development of requirement linking matrix based on semantic similarity for robust software development,” *Journal of Systems and Software*, vol. 186, p. 111211, 2022.
- [25] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, and R. T. Batista-Navarro, “Natural language processing for requirements engineering: A systematic mapping study,” *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–41, 2021.

- [26] R. Sonbol, G. Rebdawi, and N. Ghneim, “The use of NLP-based text representation techniques to support requirement engineering tasks: A systematic mapping review,” *IEEE Access*, vol. 10, pp. 62811–62830, 2022.
- [27] F. Ariai, J. Mackenzie, and G. Demartini, “Natural language processing for the legal domain: A survey of tasks, datasets, models, and challenges,” *ACM Computing Surveys*, vol. 58, no. 6, pp. 163:1–163:37, 2025.
- [28] E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer, “SecureBERT: A domain-specific language model for cybersecurity.” *Security and Privacy in Communication Networks (SecureComm)*, 2022.
- [29] K. Ethayarajh, “How contextual are contextualized word representations? comparing the geometry of BERT, ELMo, and GPT-2 embeddings,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 55–65, Association for Computational Linguistics, 2019.
- [30] S. Jamshidi, N. Shahabi, A. Nikanjam, K. W. Nafi, F. Khomh, and C. Fung, “The role of large language models in IoT security: A systematic review of advances, challenges, and opportunities,” *Internet of Things*, vol. 34, p. 101735, 2025.
- [31] K. Sapkota, A. Aldea, M. Younas, D. A. Duce, and R. Banares-Alcantara, “Automating the semantic mapping between regulatory guidelines and organizational processes,” *Service Oriented Computing and Applications*, vol. 10, no. 4, pp. 365–389, 2016.
- [32] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [33] J. R. Landis and G. G. Koch, “The measurement of observer agreement for categorical data,” *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

Glossary and Symbols

Anisotropy	Property of an embedding space in which vectors occupy a narrow cone, so that cosine similarity between arbitrary items is high regardless of meaning.
BERT	Bidirectional Encoder Representations from Transformers; a pretrained contextual language model.
Bootstrap	Nonparametric resampling procedure used to estimate the sampling distribution of a metric, here for confidence intervals.
Cohen’s kappa (κ)	Chance-corrected agreement measure between two raters on categorical labels.
Cosine similarity	Similarity of two vectors measured as the cosine of the angle between them; 1 means identical direction.
ETSI	European Telecommunications Standards Institute; publisher of the two standards studied.
F1 score	Harmonic mean of precision and recall.
Ground truth	The reference annotation against which automated predictions are evaluated; here produced with LLM assistance under a written codebook, and reviewed by the authors for the pilot set.
IoT	Internet of Things; network-connected consumer and industrial devices.
LLM	Large Language Model; a generative transformer model used here for direct relationship classification.
Macro-F1	Unweighted mean of per-class F1 scores; treats rare and frequent classes equally.
Modality	The obligation level of a provision: shall (mandatory), should (recommended), or may (permitted).
NLP	Natural Language Processing.
Provision	A single numbered normative requirement in an ETSI standard.
SBERT	Sentence-BERT; a Siamese BERT architecture trained so that sentence-level cosine similarity reflects semantic relatedness.
TF-IDF	Term Frequency-Inverse Document Frequency; a lexical text weighting scheme.

A | Relationship Label Definitions

The definitions below were used both for ground-truth annotation and in the prompt of the LLM classification method. a denotes a provision from ETSI EN 303 645, b a provision from ETSI EN 304 223.

Equivalence

a and b express the same security objective with comparable scope. Compliance with one gives strong evidence of compliance with the other. *Decision aid:* if an auditor would accept evidence for a as evidence for b without further argument, the pair is equivalent.

Subsumption (A broader)

a 's objective fully contains b 's: everything b requires is required by a , but a requires more. *Decision aid:* compliance with a implies compliance with b , not conversely.

Subsumption (B broader)

The mirror case: compliance with b implies compliance with a .

Overlap

The objectives intersect: part of what a requires is also required by b , but each provision also covers ground the other does not. Evidence can be partially reused; a gap analysis is still needed for the uncovered parts.

Complementarity

a and b address different aspects of a common security goal without overlapping scope — for example, one requires a technical mechanism and the other requires user-facing guidance about it. Implementing both is what produces the protection; neither substitutes for the other.

No relation

No meaningful connection between the objectives. Used both for obviously unrelated pairs and — more importantly for evaluation — for pairs that share vocabulary but not objective.

Annotation rules

Each pair receives exactly one label, a one-sentence justification, and a confidence score: 1 (uncertain), 2 (reasonably confident), 3 (certain). When several labels seem defensible, the annotator picks the most specific one that the provision *texts* (not assumed intent) support, and records the doubt in the justification.

B | Expert Validation Survey Instrument

This appendix contains the complete instrument and analysis plan for the proposed expert validation of the ground truth (Section 3.4.3). The study was designed but not executed within the degree project; it is specified here so it can be run without further design work. A distribution-ready copy is maintained in the project repository (`docs/expert_survey.md`).

B.1 Purpose

Measure (i) how consistently practising cybersecurity professionals assign the six relationship labels to provision pairs, and (ii) how well their majority judgement agrees with the project's ground truth. High agreement validates the ground truth as an evaluation reference; disagreement localises contestable annotations.

B.2 Participants and recruitment

Target group: professionals with at least two years of experience in cybersecurity engineering, compliance/GRC, or security assurance, familiar with at least one of the two standards or an equivalent framework (ISO/IEC 27001, IEC 62443). Target sample: 5–10 respondents recruited through professional networks and university industry contacts. Participation is anonymous and uncompensated.

B.3 Ethics and data handling

No personal data is collected beyond role category, years of experience, and self-rated familiarity. Responses are stored pseudonymously (R1, R2, ...). Participants consent via a checkbox after reading a description of the study purpose and the intended publication of aggregated results. The design collects anonymous professional judgements and no sensitive data; supervisor confirmation is obtained before deployment.

B.4 Screening questions

- S1. Current role: security engineer / compliance-GRC / auditor / researcher / other (single choice).
- S2. Years of professional experience in cybersecurity (number; respondents under 2 years excluded from the main analysis).
- S3. Familiarity with ETSI EN 303 645 (Likert 1–5).
- S4. Familiarity with ETSI EN 304 223 or other AI-security guidance (Likert 1–5).
- S5. Familiarity with compliance or standards mapping as an activity (Likert 1–5).

B.5 Main task

Each respondent rates the same 30 provision pairs, drawn from the annotated sample and stratified over all six labels, in per-respondent random order. For each pair the full text of both provisions is shown together with the label definitions of Appendix A, and the respondent answers:

Q1. **Relationship** — exactly one of the six labels.

Q2. **Confidence** — 1 (guessing) to 5 (certain).

Q3. **Justification** — optional free text.

Estimated completion time is 35–45 minutes. One duplicated pair with reversed presentation order serves as an attention check; inconsistent answers flag the respondent for review rather than automatic exclusion.

B.6 Post-task questions

P1. Clarity of the six label definitions (Likert 1–5).

P2. Which pair of labels was hardest to distinguish? (choice among label pairs).

P3. Would a tool proposing candidate mappings with these labels help your compliance work? (Likert 1–5 plus free text).

B.7 Analysis plan

1. **Inter-expert agreement:** Fleiss' kappa over all respondents, on the six-label and the merged five-label scheme, interpreted with the Landis–Koch bands [33].
2. **Expert vs. ground truth:** per-item majority vote compared with the stored label; percent agreement and Cohen's kappa [32]. Items without a majority are reported as contested.
3. **Confidence:** mean confidence per label; point-biserial correlation between low confidence and disagreement.
4. **Ground-truth revision:** items where the expert majority disagrees with the stored label are re-examined; convincing corrections enter the ground truth through a documented change log, and the full evaluation is re-run to test whether the method ranking of Chapter 5 is stable under the revised reference.

B.8 Expected outcomes

Based on comparable annotation studies, moderate inter-expert agreement ($\kappa \approx 0.4$ – 0.6) is expected on the six-label scheme, higher after merging, with the overlap/complementarity boundary as the main source of disagreement. The ground truth is considered validated for its purpose if majority agreement reaches at least 70% on the merged scheme.

C | Reproduction Guide

All results in this thesis are regenerated by the following procedure. The project requires Python 3.13, the packages pinned in `requirements.txt` (scikit-learn 1.8, pandas 3.0, sentence-transformers 5.4, transformers 5.8, torch 2.11, google-genai 2.0, matplotlib 3.11), and Poppler for `pdftotext`.

Setup

```
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
brew install poppler          # macOS; apt install poppler-utils on Debian
```

Pipeline

Run from the repository root, in order:

```
python3 -m src.extraction.provision_extractor    # ~1 s
python3 -m src.baseline.create_baseline         # <1 s
python3 -m src.mapping.rule_based               # ~4 s
python3 -m src.mapping.sbert_mapping            # ~1 min
python3 -m src.mapping.bert_mapping             # ~2 min
python3 -m src.mapping.securebert_mapping       # ~3 min
python3 -m src.evaluation.evaluate              # ~1 s
python3 -m src.evaluation.analysis              # ~1 min
python3 -m src.report.figures                   # ~10 s
```

First runs of the transformer stages download the models from Hugging Face (SecureBERT is about 500 MB). The two Gemini stages,

```
python3 -m src.mapping.gemini_embedding_mapping
python3 -m src.mapping.gemini_mapping
```

require a `GEMINI_API_KEY` in `.env` and incur API cost; the repository ships their stored outputs, which the evaluation consumes directly. The run-to-run agreement figure of Table 5.5 needs a second pass of the classifier over byte-identical prompts, written to a separate file:

```
GEMINI_OUTPUT_SUFFIX=rep2 python3 -m src.mapping.gemini_mapping
```

Probability sample and reliability

```
python3 -m src.baseline.sample_reference    # seed 20260804
python3 -m src.baseline.annotate_sample status
python3 -m src.evaluation.weighted_metrics # corpus estimates
python3 -m src.validation.agreement        # kappa + table
```

The 19 annotation batches are stored in `data/annotation/`, so these steps reproduce the reported estimates without re-annotating. Rebuilding the reference set from scratch instead requires the `emit` and `ingest` subcommands of `annotate_sample` once per batch.

Building the report

```
python3 -m src.report.check_thesis_numbers # guard: tables vs. data
cd thesis
pdflatex main.tex && bibtex main && pdflatex main.tex && pdflatex main.tex
```

The guard re-reads every number printed in a results table directly from the stored evaluation files and exits non-zero on any mismatch. It is run before each submission build, and caught three transcription errors during writing.

Determinism

Sampling and bootstrap procedures use fixed seeds (7 and 42). A verification re-run during the project reproduced the stored evaluation summary exactly. Individual embedding scores can differ in the fourth decimal between runs (CPU thread scheduling inside the tensor library); the differences are far below the label thresholds and leave every reported metric unchanged. API-based results depend on hosted model versions and are preserved as stored outputs instead.